Deep RL and Policy Optimization on CartPole v1

- Environment: CartPole v1, Gymnasium
- Seed used for every run: 0
- Training budget: 200,000 environment steps per algorithm, exactly as specified
- Evaluation: every 20,000 steps, 20 episodes, no exploration during evaluation

1 Conceptual Explanation

DQN

- Swaps the Q table for a neural network $Q_\theta(s, a)$, trained toward the target $y = r+\gamma \max_{a'} Q_{\theta^-}(s', a')$, where θ^- belongs to a separate target network.
- Replay buffer holds past transitions and samples them randomly for each update, so the network is not just learning from whatever happened in the last few steps.
- Target network is a frozen, periodically copied version of Q_θ. If you skip this and just bootstrap off the live network, the target keeps shifting under you every step, which is what our ablation actually shows.
- Exploration is epsilon greedy, epsilon decaying linearly from 1.0 down to a small floor.
- Why it can go unstable: function approximation plus bootstrapping plus sampled updates all at once means small errors in Q_θ get amplified through the max operator, and a single update can quietly mess up Q values at states that were never even visited.

Actor Critic and A2C

- Two networks (or two heads on one trunk): actor $\pi_\theta(a \mid s)$ picks actions, critic $V_\phi(s)$ scores how good the current state is.
- Advantage $A_t = G_t - V_\phi(s_t)$ tells you whether the action taken did better or worse than the critic expected.
- Actor is pushed by $-\log \pi_\theta(a_t \mid s_t) \cdot A_t$, critic is trained with a plain regression loss toward the return.
- Entropy bonus stops the policy from locking onto one action before it has actually explored enough.
- Compared to DQN: A2C is on policy, so it throws away data after one use instead of keeping a buffer, learns a stochastic policy directly instead of taking an argmax over Q values, and updates from short rollouts (5 steps across 4 parallel environments here) rather than single transitions.

PPO

- Still actor critic, but adds a mechanism to stop the policy from moving too far in one update.

- Keeps the log probability of each action from the policy that collected the rollout ($\pi_{\theta_{old}}$) and computes the ratio $r_t = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{old}}(a_t \mid s_t)$ once it starts updating on that data.
- Clipped objective: $\min(r_t A_t,\ \text{clip}(r_t, 1-\epsilon, 1+\epsilon)A_t)$. This kills the incentive to push r_t far from 1 in the direction that would help the objective, but still lets a bad step get corrected.
- Advantages come from GAE, get normalized per minibatch, and each rollout is reused across several epochs of minibatch updates instead of one pass.
- In theory this is more stable than plain A2C because the clip caps how far a single update can move the policy, whereas A2C's step size is basically just whatever the learning rate gives you.

GRPO (reading only)

- Samples multiple outputs for the same query, scores each with a reward, and uses the group's mean reward as the baseline instead of a learned critic.
- Makes sense for LLM style RL where you get a reward per full sequence from a preference or verifier model, not a dense per step signal like in CartPole.
- Not implemented, reading only per the assignment.

2 Implementation

Networks

- Every algorithm uses the same MLP shape: 2 hidden layers, 128 units each, ReLU activations, as specified in the reference setup.
- DQN: one Q network outputting a Q value per action, plus a target network that is hard copied from the online network every 500 environment steps.
- A2C / PPO: shared trunk feeding two heads, a categorical policy head (actor) and a scalar value head (critic).

Replay / Rollout Design

- DQN: circular buffer of 50,000 transitions, minibatch of 64, updates only start once the buffer has at least 1,000 transitions, then one gradient step per environment step.
- A2C: 4 vector environments, 5 step rollouts (20 transitions per update), no data reuse, returns bootstrapped off the critic at the last state.
- PPO: 4 vector environments, 128 step rollouts (512 transitions per update), GAE for advantages, then split into minibatches of 128 and reused for 4 epochs.

Losses

- DQN: MSE between $Q_\theta(s, a)$ and the bootstrapped target.
- A2C: policy loss $-E[\log \pi_\theta(a \mid s) \cdot A]$, value loss is MSE, total loss adds 0.5 times value loss and subtracts 0.01 times entropy.
- PPO: same weighting as A2C but with the clipped surrogate policy loss instead of the plain one. Gradients clipped to a max norm of 0.5 for both A2C and PPO.

Hyperparameters

Setting	DQN	A2C	PPO
Learning rate	1e-3	7e-4	3e-4
Discount γ	0.99	0.99	0.99
Batch / minibatch	64	20 (full rollout)	128
Rollout / buffer	buffer 50,000	5 steps $\times$ 4 envs	128 steps $\times$ 4 en
Update rule	1 step per env step	1 step per rollout	4 epochs of miniba
Entropy coef	n/a	0.01 (0 in ablation)	0.01
Value coef	n/a	0.5	0.5
Grad clip	n/a	0.5	0.5
Stabilizer	target net, hard update every 500 steps	n/a	clip 0.2 (0.1, 0.3 in ablatio

- No deviations from the reference setup, everything above matches the brief.

Evaluation Setup

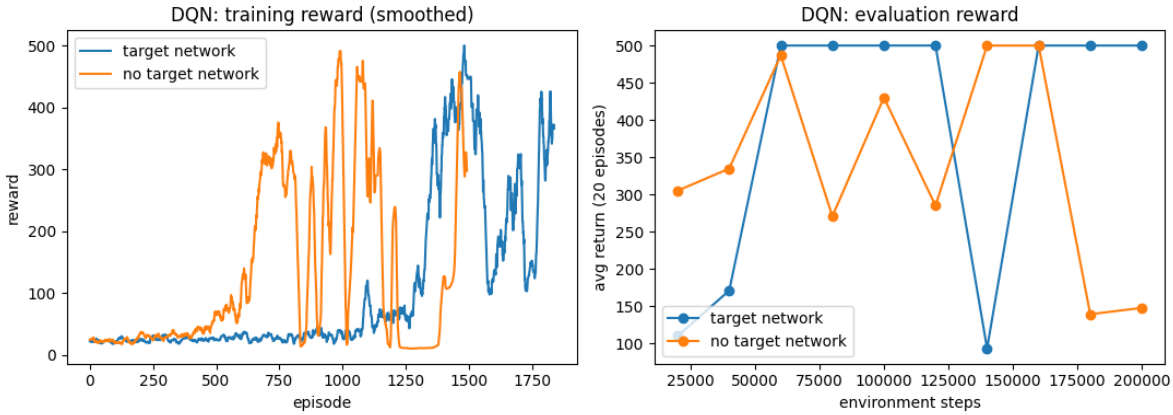
- Every 20,000 environment steps, training pauses and the current policy runs for 20 episodes on a clean environment with no exploration.
- DQN acts greedily (argmax Q). A2C and PPO take the highest probability action instead of sampling.
- Mean return over those 20 episodes is what gets logged for that step.

3 Results

Final Evaluation Reward (seed 0)

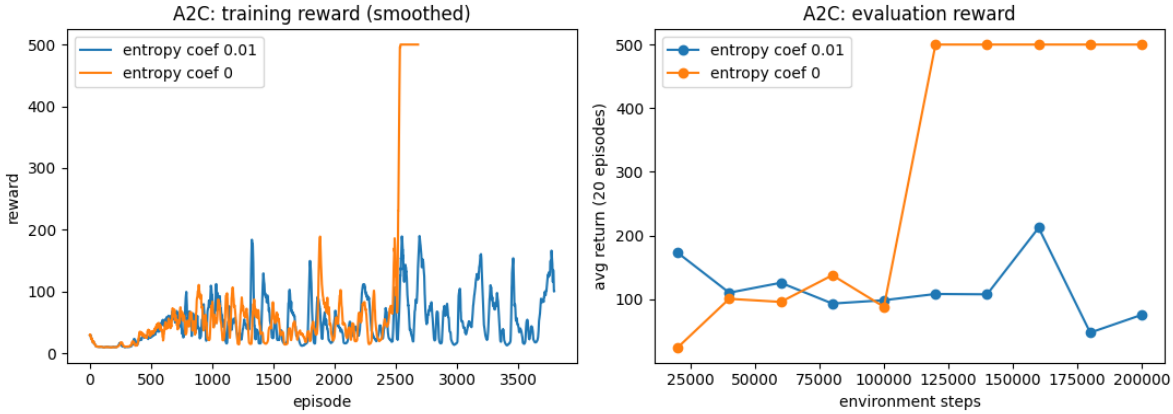
Run	Final eval reward (mean over 20 eps)
DQN, with target network	500.0
DQN, no target network	147.65
A2C, entropy coef 0.01	75.5
A2C, entropy coef 0	500.0
PPO, clip 0.1	214.2
PPO, clip 0.2	418.65
PPO, clip 0.3	354.0

DQN: target network vs no target network



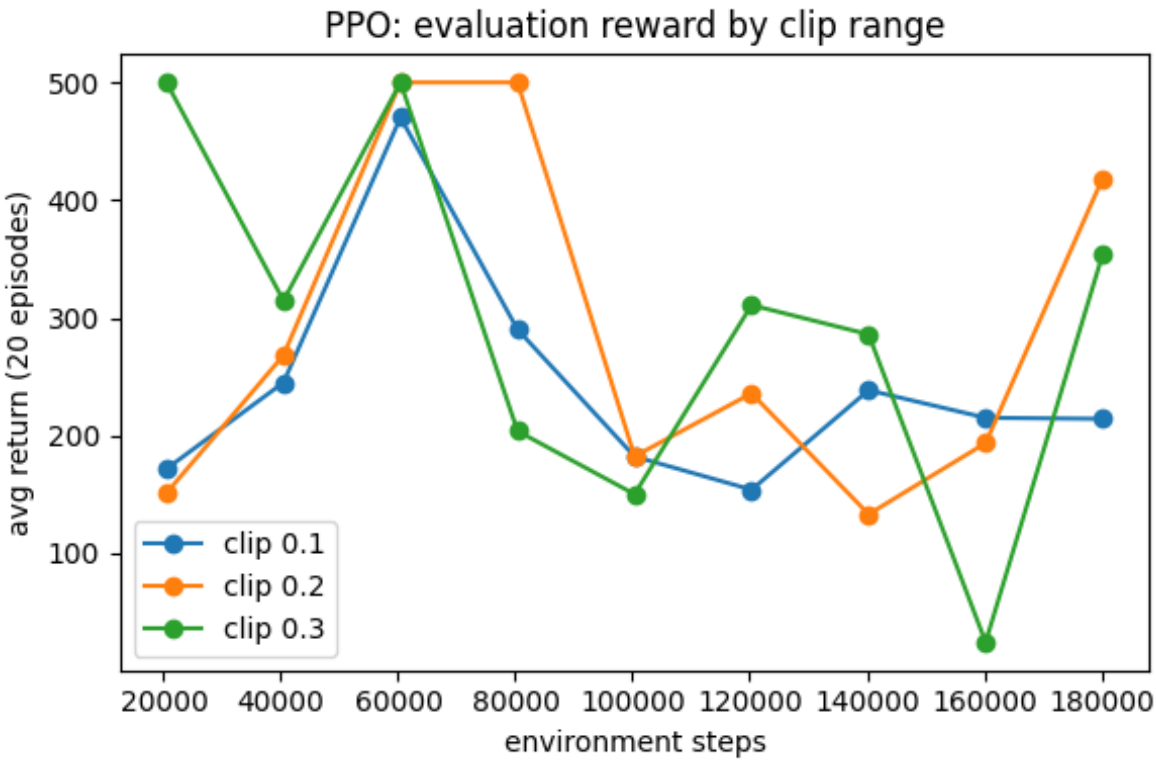
- With the target network, DQN reaches the max reward of 500 comfortably.
- Without it, training is clearly worse and noisier, and it settles far below max at 147.65.

A2C: entropy coefficient ablation



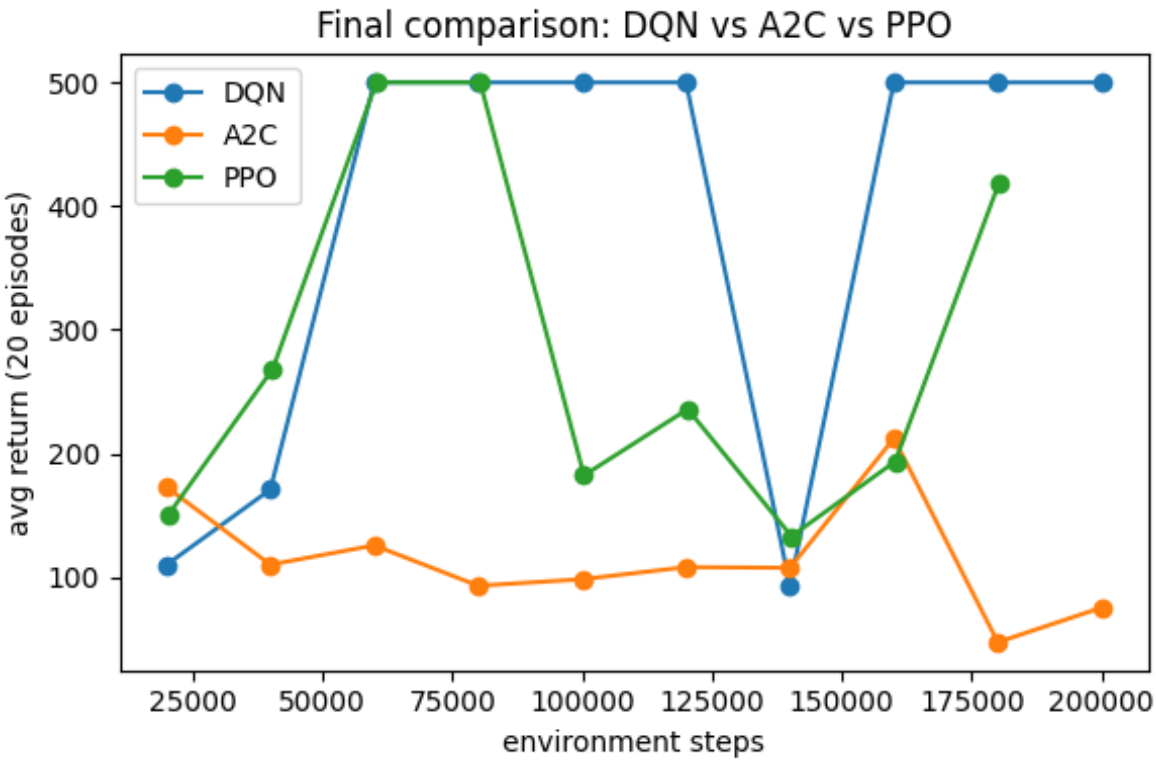
- Entropy 0 actually hit the max reward of 500, entropy 0.01 only reached 75.5.
- This is the opposite of what we expected going in, more on this in the discussion below.

PPO: clip range ablation



- Clip 0.2 (default) was the best of the three at 418.65.
- Clip 0.3 came second at 354.0, clip 0.1 was worst at 214.2.

Final Comparison: DQN vs A2C vs PPO



4 Discussion

Convergence and instability

- DQN with the target network converges cleanly to 500, which matches the theory, the frozen target keeps the bootstrapped label stable long enough for the network to actually learn it.
- Removing the target network drops final performance to 147.65. The online network is now chasing a target that moves every single update, so it never really settles, exactly the moving target problem we described in section 1.
- PPO is fairly stable across all three clip values, none of the runs collapse, they just land at different final rewards. That is roughly what we would expect since the clip is there to bound how far each update can move the policy, not to guarantee the best possible policy.

Ablation observations

- DQN ablation is the cleanest result in the whole assignment, removing the target network clearly hurts performance, which is exactly the point of that experiment.
- A2C ablation gave a result we did not expect. Entropy 0.01 should if anything help exploration and stability, but it scored much worse than entropy 0 on this single seed. With only 5 step rollouts and a single seed, A2C's variance is high, so this is most likely seed and run to run noise rather than entropy actively hurting. This is worth flagging rather than hiding since the assignment only requires one seed.

- PPO clip ablation makes sense on its face, clip 0.1 is the most conservative and ends up the slowest to improve within a fixed 200,000 step budget, while clip 0.2 and 0.3 are large enough to make faster progress in the same budget. Clip 0.2 doing best suggests it is a decent middle ground for CartPole specifically.

Comparing the three algorithms

- Simplest to implement: DQN. Its ablation is also the most interpretable, target network on or off is basically a switch.
- Most stable across ablations: PPO, all three clip settings stay in a reasonable range without falling apart.
- Most sensitive to setup in our runs: A2C, its entropy ablation flipped the result we expected, which lines up with it being the highest variance algorithm of the three (short rollouts, no clipping, no replay buffer to smooth things out).
- Sample efficiency at 200,000 steps: DQN (with target net) and A2C (entropy 0) both hit the ceiling of 500, PPO's best run got to 418.65, so PPO did not fully converge in this budget even though it stayed stable, which is a fair trade off given the other two algorithms are less forgiving to implement correctly.

5 References

- Sutton and Barto, Reinforcement Learning: An Introduction, 2nd ed. https://incompleteideas.net/book/the-book-2nd.html
- Mnih et al., Playing Atari with Deep Reinforcement Learning (DQN). https://arxiv.org/abs/1312.5602
- Mnih et al., Asynchronous Methods for Deep Reinforcement Learning (A3C). https://arxiv.org/abs/1602.01783
- Schulman et al., Proximal Policy Optimization Algorithms. https://arxiv.org/abs/1707.06347
- OpenAI Spinning Up, PPO. https://spinningup.openai.com/en/latest/algorithms/ppo.html
- Hugging Face Deep RL Course. https://huggingface.co/learn/deep-rl-course/
- David Silver RL Course. https://www.davidsilver.uk/teaching/
- Gymnasium CartPole v1 docs. https://gymnasium.farama.org/environments/classic_control/cart_pole/
- Gymnasium LunarLander v3 docs. https://gymnasium.farama.org/environments/box2d/lunar_lander/
- Shao et al., DeepSeekMath (GRPO). https://arxiv.org/abs/2402.03300